

CYBER SECURITY

1. BASIC NETWORK SNIFFER NETWORK PACKET CAPTURE AND ANALYSIS

This project was completed as part of CodeAlpha Cyber Security Internship - Task 1. It captures live IPv4 traffic with Python and Scapy, extracts key packet fields, and analyzes TCP, UDP, ICMP, DNS, and payload data.

Introduction and Objective

The objective is to build a basic packet sniffer, understand packet structure through practical testing, and display source/destination IP addresses, protocols, ports, DNS queries/responses, and payload data from live traffic. Npcap is used for packet capture on Windows.

Document type: Technical Report / Network Security

Scope: Packet Capture & Traffic Analysis

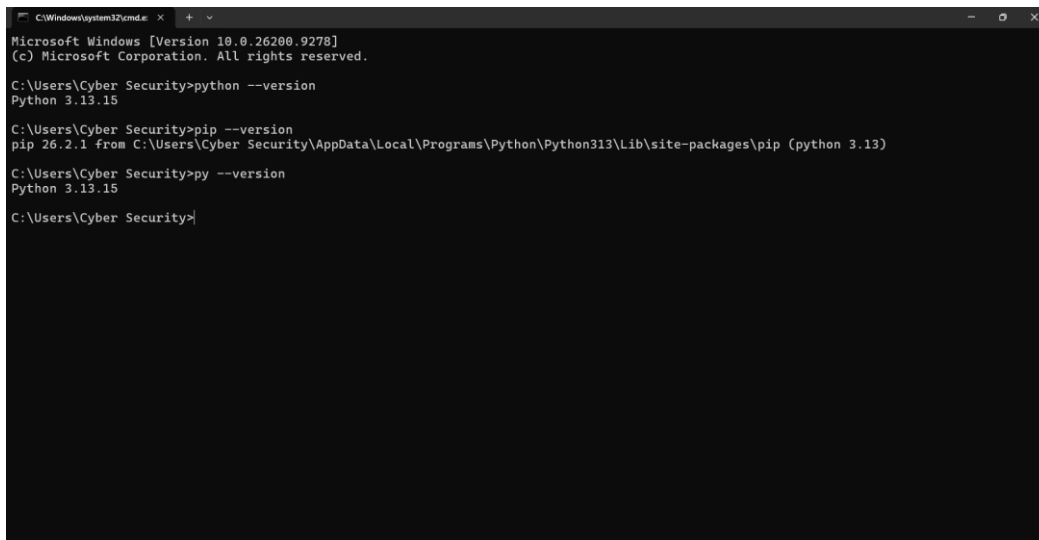
Author / Role: Murad Mahmudov | Cyber Security Intern

Date: 14 September 2026

GitHub: https://github.com/muradmahmudoff/CodeAlpha__tasks/tree/main/Task1_NetworkSniffer

1. Python Environment and Project Setup

Initial setup of the Python interpreter, virtual environment, and project directory



```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.26200.9278]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Cyber Security>python --version
Python 3.13.15

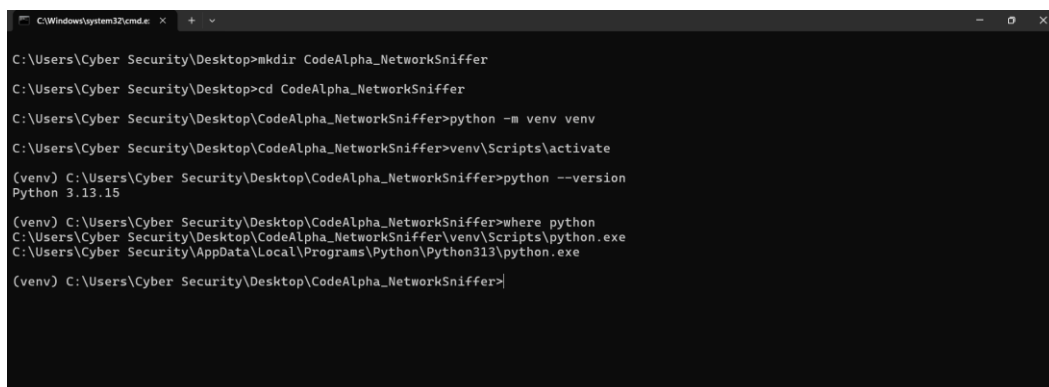
C:\Users\Cyber Security>pip --version
pip 26.2.1 from C:\Users\Cyber Security\AppData\Local\Programs\Python\Python313\Lib\site-packages\pip (python 3.13)

C:\Users\Cyber Security>py --version
Python 3.13.15

C:\Users\Cyber Security>
```

Figure 1. Verifying Python and pip versions

After installing Python 3.13.15 and pip 26.2.1, both versions were verified from the command line.



```
C:\Users\Cyber Security\Desktop>mkdir CodeAlpha_NetworkSniffer
C:\Users\Cyber Security\Desktop>cd CodeAlpha_NetworkSniffer
C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>python -m venv venv
C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>venv\Scripts\activate
(venv) C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>python --version
Python 3.13.15
(venv) C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>where python
C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer\venv\Scripts\python.exe
C:\Users\Cyber Security\AppData\Local\Programs\Python\Python313\python.exe
(venv) C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>
```

Figure 2. Creating and activating the virtual environment

The CodeAlpha_NetworkSniffer directory was created, the venv environment was activated, and the local Python interpreter was verified.

2. Npcap and Scapy Packet Capture Environment

Installing the Windows packet capture driver and Python analysis library

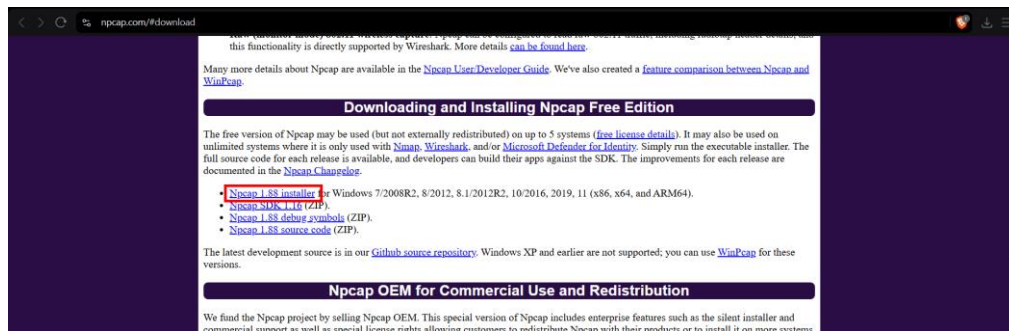


Figure 3. Selecting the Npcap installer from the official source

Npcap 1.88 was downloaded from the official source to provide packet capture support on Windows.

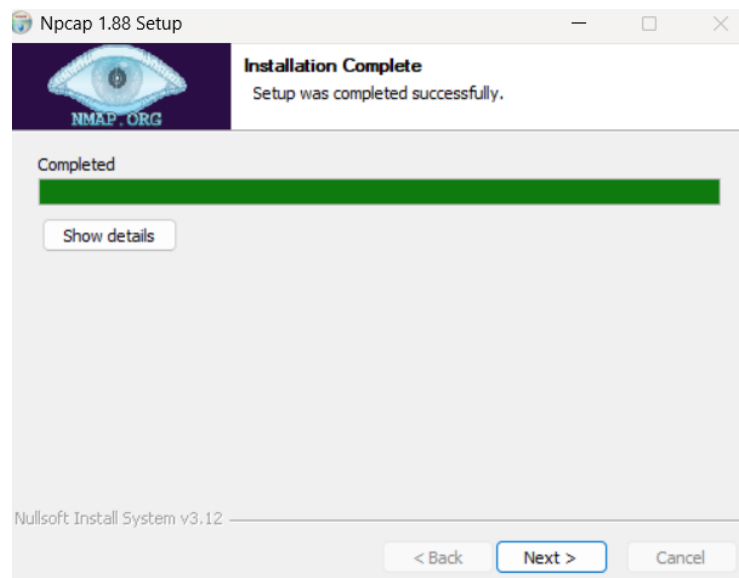


Figure 4. Completing the Npcap installation

Npcap setup completed successfully, providing the capture driver required by Scapy.

```
C:\Windows\system32\cmd.exe
(venv) C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>python -m pip install scapy
Collecting scapy
  Downloading scapy-2.7.0-py3-none-any.whl.metadata (5.8 kB)
  Downloading scapy-2.7.0-py3-none-any.whl (2.6 MB)
    2.6/2.6 MB 7.1 MB/s 0:00:00
Installing collected packages: scapy
Successfully installed scapy-2.7.0

(venv) C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>python -c "import scapy; print('Scapy version:', scapy.__version__)"
Scapy version: 2.7.0

(venv) C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>python -m pip show scapy
Name: scapy
Version: 2.7.0
Summary: Scapy: interactive packet manipulation tool
Home-page: https://scapy.net
Author: Philippe BIONDI, Gabriel POTTER
Author-email:
License: GPL-2.0-only
Location: C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer\venv\Lib\site-packages
Requires:
Required-by:
```

Figure 5. Installing the Scapy library

Scapy 2.7.0 was installed inside the virtual environment and its version was verified.

3. Initial Sniffer and Live Packet Capture

Building the first packet capture flow with Scapy sniff()

```
sniffer.py
File Edit View
from scapy.all import sniff

def packet_callback(packet):
    print(packet.summary())

print("Starting network sniffer...")
print("Press Ctrl+C to stop.\n")
sniff(prn=packet_callback, store=False)
```

Figure 6. Creating the initial sniffer code

The first version sends every captured packet to a callback function and displays a short summary with packet.summary().

```
Administrator: C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.26200.9278]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>cd "%USERPROFILE%\Desktop\CodeAlpha_NetworkSniffer"

C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>venm\Scripts\activate

(venv) C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>python sniffer.py
Starting network sniffer...
Press Ctrl+C to stop.

Ether / IP / TCP 192.168.100.130:55918 > 20.215.74.200:https PA / Raw
Ether / IP / TCP 20.215.74.200:https > 192.168.100.130:55918 A / Padding
Ether / IP / TCP 192.168.100.130:24234 > 104.18.39.21:https PA / Raw
Ether / Dot1Q / IP / TCP 104.18.39.21:https > 192.168.100.130:24234 A / Padding
Ether / Dot1Q / IP / TCP 104.18.39.21:https > 192.168.100.130:24234 PA / Raw
Ether / IP / TCP 192.168.100.130:24234 > 104.18.39.21:https A
Ether / IP / TCP 192.168.100.130:49409 > 98.66.133.184:https PA / Raw
Ether / IP / TCP 98.66.133.184:https > 192.168.100.130:49409 PA / Raw
Ether / IP / TCP 192.168.100.130:49409 > 98.66.133.184:https A
Ether / IP / TCP 20.215.74.200:https > 192.168.100.130:55918 PA / Raw
Ether / IP / TCP 192.168.100.130:55918 > 20.215.74.200:https A
Ether / IP / TCP 192.168.100.130:24234 > 104.18.39.21:https PA / Raw
Ether / IP / TCP 192.168.100.130:41508 > 172.64.155.209:https PA / Raw
Ether / IP / TCP 192.168.100.130:41508 > 172.64.155.209:https PA / Raw
Ether / IP / TCP 192.168.100.130:41508 > 172.64.155.209:https PA / Raw
Ether / Dot1Q / IP / TCP 104.18.39.21:https > 192.168.100.130:24234 A / Padding
Ether / Dot1Q / IP / TCP 172.64.155.209:https > 192.168.100.130:41508 A / Padding
Ether / Dot1Q / IP / TCP 172.64.155.209:https > 192.168.100.130:41508 A / Padding
Ether / Dot1Q / IP / TCP 172.64.155.209:https > 192.168.100.130:41508 A / Padding
Ether / Dot1Q / IP / TCP 172.64.155.209:https > 192.168.100.130:41508 A / Padding
Ether / Dot1Q / IP / TCP 172.64.155.209:https > 192.168.100.130:41508 PA / Raw
Ether / IP / TCP 192.168.100.130:41508 > 172.64.155.209:https A
Ether / Dot1Q / IP / TCP 104.18.39.21:https > 192.168.100.130:24234 PA / Raw
Ether / IP / TCP 192.168.100.130:24234 > 104.18.39.21:https A
```

Figure 7. Capturing live network traffic for the first time

In an Administrator session, TCP, UDP, DNS, and other live network packets were captured successfully.

4. Extended Packet Structure Analysis

Extracting source/destination IP, ports, protocol, and payload information

```
sniffer.py
File Edit View
from scapy.all import sniff, IP, TCP, UDP, ICMP, Raw

def packet_callback(packet):
    # Only analyze IPv4 packets
    if IP not in packet:
        return

    source_ip = packet[IP].src
    destination_ip = packet[IP].dst

    print("-" * 70)
    print(f"Source IP : {source_ip}")
    print(f"Destination IP : {destination_ip}")

    # TCP packet
    if TCP in packet:
        print(f"Protocol : TCP")
        print(f"Source Port : {packet[TCP].sport}")
        print(f"Destination Port: {packet[TCP].dport}")

    # UDP packet
    elif UDP in packet:
        print(f"Protocol : UDP")
        print(f"Source Port : {packet[UDP].sport}")
        print(f"Destination Port: {packet[UDP].dport}")

    # ICMP packet
    elif ICMP in packet:
        print(f"Protocol : ICMP")

    else:
        print(f"Protocol Number : {packet[IP].proto}")

    # Display payload if present
    if Raw in packet:
        payload = bytes(packet[Raw].load)
        print(f"Payload Length : {len(payload)} bytes")
        print(f"Payload Preview : {payload[:80]}r")
    else:
        print(f"Payload : No Raw payload")

    print("-" * 70)

print("CodeAlpha Basic Network Sniffer")
print("Capturing network packets...")
print("Press Ctrl+C to stop.")

sniff(prn=packet_callback, store=False)

in 50 Cpu 4.134 Gbps 100% Windows (C) P. UTR 8
```

Figure 8. Structured packet analysis code

The code filters IPv4 packets and displays TCP, UDP, and ICMP protocols together with IP addresses, ports, and Raw payload data.

```
Administrator: C:\Windows\system32\cmd.exe
(c:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>python sniffer.py
CodeAlpha Basic Network Sniffer
Capturing network packets...
Press Ctrl+C to stop.

=====
Source IP      : 192.168.100.130
Destination IP : 172.64.155.209
Protocol       : TCP
Source Port    : 41508
Destination Port: 443
Payload Length : 9070 bytes
Payload Preview: b'\x17\x03\x03\x1f\x05\x09\xad\x9f\x94\x06\xfc\x02\x03\x0a\x0a\x07\x07\x05\x07\x0b\x0e\x04\x01\x03\x0e\xfb\x18\xfe\x01\xff\xff\xff\x03\x00\xbc\x1d\x03\x00\xcb\x1e\x04\x0b\x0e\x03\x09\x00\x15\x08\x0a\x02\x03\x02'
=====
Source IP      : 192.168.100.130
Destination IP : 172.64.155.209
Protocol       : TCP
Source Port    : 41508
Destination Port: 443
Payload Length : 14872 bytes
Payload Preview: b'\x17\x03\x03:\x13,cw0t\x00\x1f\x08\x0b\x04\xef\x04\x04\xff\x0b\x12\x070)-\x0c\x070k\x02-g\x0d0w\x121v\x0f005-7\x0c\x06\x0a\x0f\x02\xce\x00\x01\x0c43\x05\x0c9v\x0f\x085q\x0f f\x0ae-8e\x07+-\x0b0'
=====
Source IP      : 172.64.155.209
Destination IP : 192.168.100.130
Protocol       : TCP
Source Port    : 443
Destination Port: 41508
Payload        : No Raw payload
=====
Source IP      : 172.64.155.209
Destination IP : 192.168.100.130
Protocol       : TCP
Source Port    : 443
Destination Port: 41508
Payload        : No Raw payload
=====
```

Figure 9. TCP and HTTPS packet analysis results

Live HTTPS traffic showed source/destination IP addresses, TCP port 443, and encrypted payload preview values.

5. Testing ICMP and UDP Traffic

Verifying different protocols with dedicated test traffic

```
Administrator: C:\Windows\system32\cmd.exe
Source IP      : 192.168.100.130
Destination IP : 20.215.74.200
Protocol       : TCP
Source Port    : 55918
Destination Port: 443
Payload        : No Raw payload
=====
Source IP      : 192.168.100.130
Destination IP : 8.8.8.8
Protocol       : ICMP
Payload Length : 32 bytes
Payload Preview: b'abcdefghijklnopqrstuvwabcdefghi'
=====
Source IP      : 8.8.8.8
Destination IP : 192.168.100.130
Protocol       : ICMP
Payload Length : 32 bytes
Payload Preview: b'abcdefghijklnopqrstuvwabcdefghi'
=====
Source IP      : 192.168.100.130
Destination IP : 8.8.8.8
Protocol       : ICMP
Payload Length : 32 bytes
Payload Preview: b'abcdefghijklnopqrstuvwabcdefghi'
=====
Source IP      : 192.168.100.130
Destination IP : 192.168.100.130
Protocol       : ICMP
Payload Length : 32 bytes
Payload Preview: b'abcdefghijklnopqrstuvwabcdefghi'
=====
Source IP      : 192.168.100.130
Destination IP : 8.8.8.8
Protocol       : ICMP
Payload Length : 32 bytes
Payload Preview: b'abcdefghijklnopqrstuvwabcdefghi'
=====
Source IP      : 192.168.100.130
Destination IP : 192.168.100.130
Protocol       : ICMP
Payload Length : 32 bytes
Payload Preview: b'abcdefghijklnopqrstuvwabcdefghi'
=====
Source IP      : 8.8.8.8
Destination IP : 192.168.100.130
Protocol       : ICMP
Payload Length : 32 bytes
Payload Preview: b'abcdefghijklnopqrstuvwabcdefghi'
=====
```

Figure 10. Capturing ICMP echo request and reply packets

ICMP traffic generated with ping 8.8.8.8 showed request, reply, and 32-byte payload data.

```

Administrator: C:\Windows\system32\cmd.exe
=====
Source IP      : 192.168.100.130
Destination IP : 8.8.8.8
Protocol       : UDP
Source Port    : 62784
Destination Port: 53
Payload        : No Raw payload
=====
Source IP      : 8.8.8.8
Destination IP : 192.168.100.130
Protocol       : UDP
Source Port    : 53
Destination Port: 62784
Payload        : No Raw payload
=====
Source IP      : 192.168.100.130
Destination IP : 8.8.8.8
Protocol       : UDP
Source Port    : 62785
Destination Port: 53
Payload        : No Raw payload
=====
Source IP      : 8.8.8.8
Destination IP : 192.168.100.130
Protocol       : UDP
Source Port    : 53
Destination Port: 62785
Payload        : No Raw payload
=====
Source IP      : 192.168.100.130
Destination IP : 8.8.8.8
Protocol       : UDP
Source Port    : 62786
Destination Port: 53
Payload        : No Raw payload
=====
Source IP      : 8.8.8.8
Destination IP : 192.168.100.130
Protocol       : UDP
Source Port    : 53
Destination Port: 62786
Payload        : No Raw payload
=====

```

Figure 11. Capturing UDP port 53 DNS traffic

During the nslookup test, outbound and inbound UDP packets on port 53 to 8.8.8.8 were identified correctly.

6. DNS Query and Response Analysis

Extending UDP traffic analysis with application-layer information

```

File Edit View
from scapy.all import sniff, IP, TCP, UDP, ICMP, Raw, DNS, DNSQR

def packet_callback(packet):
    # Only analyze IPv4 packets
    if IP not in packet:
        return
    source_ip = packet[IP].src
    destination_ip = packet[IP].dst
    print("-" * 70)
    print("Source IP      : (%s)" % (source_ip))
    print("Destination IP    : (%s)" % (destination_ip))
    # TCP packet
    if TCP in packet:
        print("Protocol         : TCP")
        print("Source Port      : (%s)" % (packet[TCP].sport))
        print("Destination Port : (%s)" % (packet[TCP].dport))
    # UDP packet
    elif UDP in packet:
        print("Protocol         : UDP")
        print("Source Port      : (%s)" % (packet[UDP].sport))
        print("Destination Port : (%s)" % (packet[UDP].dport))
    # ICMP packet
    elif ICMP in packet:
        print("Protocol         : ICMP")
    # Other IP protocols
    else:
        print("Protocol Number : (%s)" % (packet[IP].proto))
    # DNS information
    if DNS in packet:
        if packet[DNS].qr == 0:
            print("DNS Type       : Query")
        else:
            print("DNS Type       : Response")
        if DNSQR in packet:
            query_name = packet[DNSQR].qname.decode(errors="replace")
            print("DNS Query      : (%s)" % (query_name))
    # Payload information
    if Raw in packet:
        payload = bytes(packet[Raw].load())
        print("Payload length : (%s) bytes" % (len(payload)))
        print("Payload Preview : (%s)" % (payload[:40]))
    else:
        print("Payload        : No Raw payload")
    print("-" * 70)

print("CodeAlpha Basic Network Sniffer")
print("Capturing network packets...")
print("Press Ctrl+C to stop.")
sniff(prn=packet_callback, store=0)

```

Figure 12. Adding DNS layer analysis to the code

Using the Scapy DNS and DNSQR layers, the program displays query/response type and the requested domain name.

```

Administrator: C:\Windows\system32\cmd.exe
=====
Source IP      : 192.168.100.130
Destination IP : 8.8.8.8
Protocol       : UDP
Source Port    : 58881
Destination Port : 53
DNS Type       : Query
DNS Query      : example.com.
Payload        : No raw payload
=====
Source IP      : 8.8.8.8
Destination IP : 192.168.100.130
Protocol       : UDP
Source Port    : 53
Destination Port : 58881
DNS Type       : Response
DNS Query      : example.com.
Payload        : No raw payload
=====
Source IP      : 192.168.100.130
Destination IP : 8.8.8.8
Protocol       : UDP
Source Port    : 53
Destination Port : 58882
DNS Type       : Query
DNS Query      : example.com.
Payload        : No raw payload
=====
Source IP      : 8.8.8.8
Destination IP : 192.168.100.130
Protocol       : UDP
Source Port    : 53
Destination Port : 58882
DNS Type       : Response
DNS Query      : example.com.
Payload        : No raw payload
=====
(Powercat) C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffers

```

Figure 13. Practical result of the example.com DNS query

Query and response packets for example.com were separated, confirming the DNS server, UDP port 53, and domain name within the same capture.

7. Final Code Structure and Graceful Shutdown

Adding packet counter, timestamp, and controlled shutdown behavior

```

sniffer.py
File Edit View
from datetime import datetime
from scapy.all import sniff, IP, TCP, UDP, ICMP, Raw, DNS, DNSQR

packet_count = 0

def packet_callback(packet):
    global packet_count

    # Only analyze IPv4 packets
    if IP not in packet:
        return

    packet_count += 1

    source_ip = packet[IP].src
    destination_ip = packet[IP].dst
    capture_time = datetime.now().strftime("%d:%h:%S")

    print(f"< * 70")
    print(f"Packet #: {packet_count}")
    print(f"Time       : {capture_time}")
    print(f"Source IP   : {source_ip}")
    print(f"Destination IP : {destination_ip}")

    # TCP packet
    if TCP in packet:
        print(f"Protocol      : TCP")
        print(f"Source Port   : {packet[TCP].sport}")
        print(f"Destination Port: {packet[TCP].dport}")

    # UDP packet
    elif UDP in packet:
        print(f"Protocol      : UDP")
        print(f"Source Port   : {packet[UDP].sport}")
        print(f"Destination Port: {packet[UDP].dport}")

    # ICMP packet
    elif ICMP in packet:
        print(f"Protocol      : ICMP")

    # Other IP protocols
    else:
        print(f"Protocol Number : {packet[IP].proto}")

    # DNS information
    if DNS in packet:
        if packet[DNS].qr == 0:
            print(f"DNS Type       : Query")
        else:

```

Figure 14. First part of the final sniffer code

The final version adds a packet counter and capture timestamp while keeping the main IPv4 packet fields in a structured format.

```

sniffer.py
File Edit View
# Other IP protocols
else:
    print(f"Protocol Number : {packet[IP].proto}")

# DNS information
if DNS in packet:
    if packet[DNS].qr == 0:
        print("DNS Type : Query")
    else:
        print("DNS Type : Response")

    if DNSQR in packet:
        query_name = packet[DNSQR].qname.decode(errors="replace")
        print(f"DNS Query : {query_name}")

# Payload information
if Raw in packet:
    payload = bytes(packet[Raw].load)
    print(f"Payload length : {len(payload)} bytes")
    print(f"Payload Preview : {payload[:80]}")
else:
    print("Payload : No Raw payload")

print("-" * 70)

def main():
    print("-" * 70)
    print("CodeAlpha Basic Network Sniffer")
    print("-" * 70)
    print("Capturing IPv4 network packets...")
    print("Press Ctrl+C to stop the sniffer.")
    print("-" * 70)
    print()

    try:
        sniff(prmpacket_callback, store=False)
    except KeyboardInterrupt:
        pass

    finally:
        print()
        print("-" * 70)
        print("Network sniffer stopped.")
        print(f"Total IPv4 packets analyzed: {packet_count}")
        print("-" * 70)

if __name__ == "__main__":
    main()

```

Figure 15. Second part of the final sniffer code

DNS and payload analysis are preserved; main() and try/finally provide a clean Ctrl+C shutdown and display the final packet count.

8. Final Execution and Project Files

Running the final program and preparing dependencies and Git files

```

Administrator: C:\Windows\system32\cmd.exe
Payload : No Raw payload
=====
Packet #22
Time : 11:36:33
Source IP : 20.52.64.201
Destination IP : 192.168.100.130
Protocol : TCP
Source Port : 443
Destination Port: 16212
Payload Length : 502 bytes
Payload Preview : b'\x17\x02\x03\x01\xf1\xe5X\xbe\xeb\x09\n"\xaa\xc3\xa2kP\xed\x01\x94\xcf\x89\x03\x9a\x0e\x07\x07\xa3\xa1\x0e\xaf\x08\xe0\x86\xf5A\x02\xf9\xcf\xbe>\x9f\xa3\xfb\xec\xf1\t\xed5\x8b\x03
Vh\x8e\xfcH\xfb8\x08\x16\xcb\xdf\xab\xbbd' "\xb32'\x0d1\x02\x99('
=====
Packet #23
Time : 11:36:33
Source IP : 192.168.100.130
Destination IP : 20.52.64.201
Protocol : TCP
Source Port : 16212
Destination Port: 443
Payload : No Raw payload
=====
Network sniffer stopped.
Total IPv4 packets analyzed: 23
=====
(venv) C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>

```

Figure 16. Final sniffer execution and packet counter result

The program numbered and analyzed packets, then displayed Total IPv4 packets analyzed after Ctrl+C.

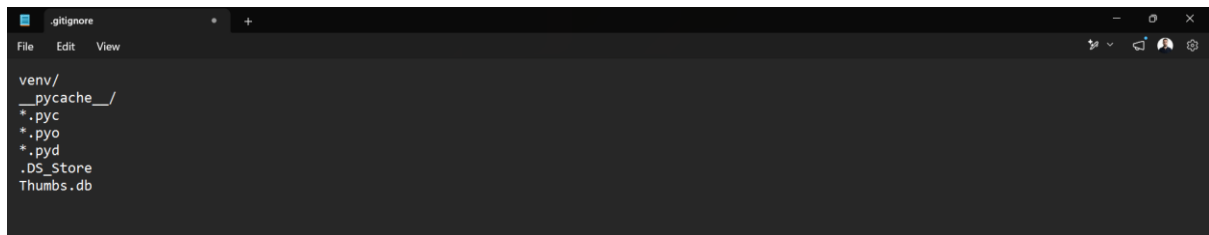
```

Administrator: C:\Windows\system32\cmd.exe
(venv) C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>python -m pip freeze > requirements.txt
(venv) C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>type requirements.txt
scapy==2.7.0
(venv) C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>

```

Figure 17. Creating the requirements.txt file

The Scapy 2.7.0 dependency was recorded in requirements.txt.

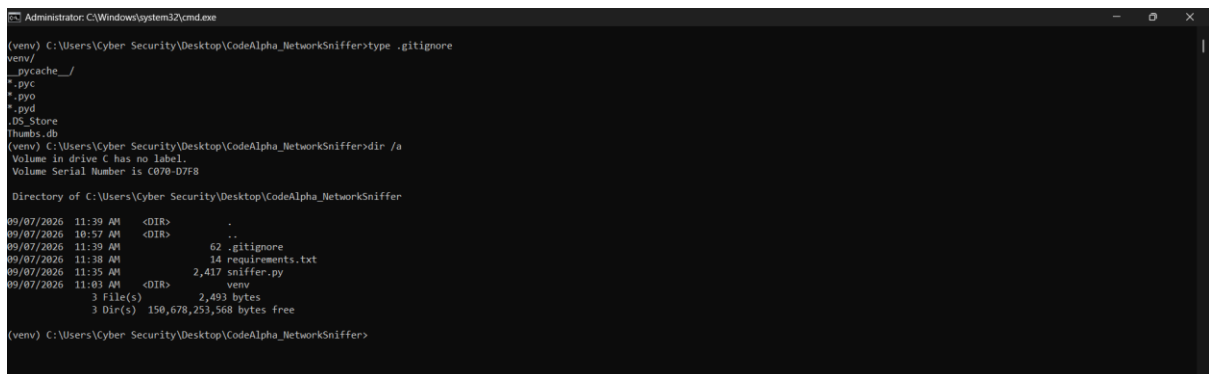


```
.gitignore
File Edit View

venv/
__pycache__/
*.pyc
*.pyo
*.pyd
.DS_Store
Thumbs.db
```

Figure 18. Preparing the .gitignore file

The virtual environment, Python cache, and temporary system files were excluded from the Git repository.



```
Administrator: C:\Windows\system32\cmd.exe

(venv) C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>type .gitignore
venv/
__pycache__/
*.pyc
*.pyo
*.pyd
.DS_Store
Thumbs.db
(venv) C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>dir /a
Volume in drive C has no label.
Volume Serial Number is C070-D7F8

Directory of C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer

09/07/2026 11:39 AM <DIR>      .
09/07/2026 10:57 AM <DIR>      ..
09/07/2026 11:39 AM          62 .gitignore
09/07/2026 11:38 AM          14 requirements.txt
09/07/2026 11:39 AM          2,417 sniffer.py
09/07/2026 11:03 AM <DIR>      venv
               3 File(s)      2,493 bytes
               3 Dir(s)  150,678,253,568 bytes free

(venv) C:\Users\Cyber Security\Desktop\CodeAlpha_NetworkSniffer>
```

Figure 19. Final verification of project files

The presence of sniffer.py, README.md, requirements.txt, .gitignore, and the screenshots directory was verified.

9. GitHub Repository and Documentation

Publishing source code and screenshot evidence in a public repository

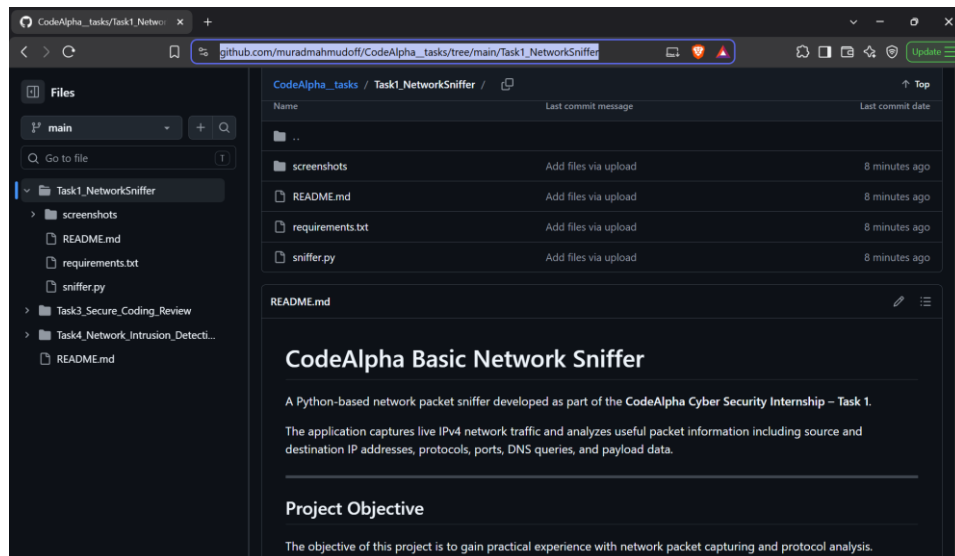


Figure 20. CodeAlpha_NetworkSniffer GitHub repository

The source code, README, dependency file, and screenshots directory were pushed to the public repository on the main branch.

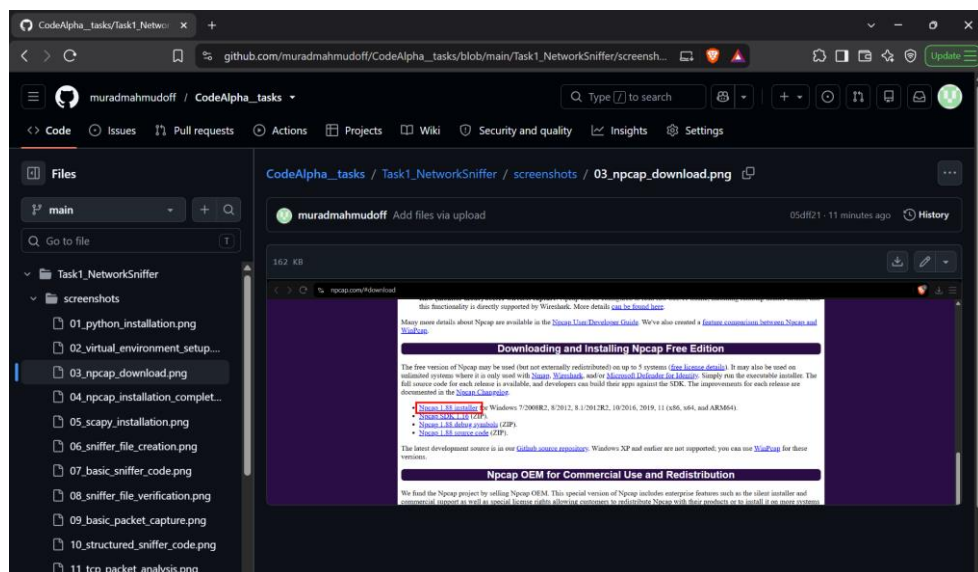


Figure 21. Verifying screenshot files on GitHub

The screenshot files were confirmed to use the correct names and open successfully from GitHub.

Repository:

https://github.com/muradmahmudoff/CodeAlpha_tasks/tree/main/Task1_NetworkSniffer

Conclusion

The project resulted in a working Basic Network Sniffer built with Python and Scapy and tested with real traffic on Windows. In line with CodeAlpha Task 1 requirements, it captures live packets and presents the main packet information in a clear format.

The final program captures live IPv4 packets, identifies TCP/UDP/ICMP traffic, displays source and destination IP/port information, recognizes DNS queries and responses, and shows a payload preview when a Raw layer is present. Packet numbering, timestamps, and a final packet count after Ctrl+C were also added. The source code, README, requirements.txt, .gitignore, and test screenshots are published in the public GitHub repository. Packet sniffing should only be performed on personal or explicitly authorized networks.